

Laboratório 02

Julia Maria Pereira Pinheiro

Nesse Laboratório, vamos determinar:

- Percentual de vôos por Cia;
- Áreas que apresentaram atrasos de chegada superiores a 10 minutos.

Com as condições de que:

- As companhia utilizadas serão: AA, DL, UA e US;
- Deve-se ler 100.000 observações por vez;
- A estatística de interesse deve ser calculada para cada um dos dias de 2015;
- Usaremos apenas o pacote `dplyr` e `readr` para determinação do percentual e importação dos dados;
- Os resultados devem ser apresentados em um formato de calendário.

Etapas

- 1) Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Para determinar o percentual de vôos atrasados na chegada, com mais de 10 minutos de atraso, precisamos fazer uma contagem dos vôos que seguem nossa condição e a contagem do total de vôos por grupo.

- 2) Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;

```

getStats <- function(input, pos){
  # item a
  input %>% filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%
  # item b
  filter(!is.na(ARRIVAL_DELAY), !is.na(YEAR), !is.na(MONTH), !is.na(DAY)) %>%
  # item c
  group_by(YEAR, MONTH, DAY, AIRLINE) %>%
  # item d
  summarise(
    atrasados = sum(ARRIVAL_DELAY > 10),
    total = n(),
    .groups = "drop"
  )
}

```

- 3) Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

colunas <- cols_only(
  YEAR = col_integer(),
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)
dados_resumidos <- read_csv_chunked(
  file = "dados/flights.zip",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = colunas
)

```

- 4) Crie uma função chamada `computeStats` que:
 - a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);

- b. Retorne as informações em um tibble contendo apenas as seguintes colunas: - **Cia**: sigla da companhia aérea; - **Data**: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); - **Perc**: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats <- function(stats) {  
  stats %>%  
  group_by(YEAR, MONTH, DAY, AIRLINE) %>%  
    summarise(  
      atrasados = sum(atrasados),  
      total = sum(total),  
      .groups = "drop"  
    ) %>%  
  mutate(  
    Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),  
    Perc = atrasados/total  
  ) %>%  
  select(Cia = AIRLINE, Data, Perc)  
}  
stats_finais <- computeStats(dados_resumidos)
```

- 5) Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
- a) Instale e carregue os pacotes `ggcal` e `ggplot2`.
 - b) Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
 - c) Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: - Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. - Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. - Retornar para o usuário a base do calendário criada acima.
 - d) Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `CAA`, `CDL`, `CUA` e `CUS`.
 - e) Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")  
  
baseCalendario <- function(stats, cia) {
```

```

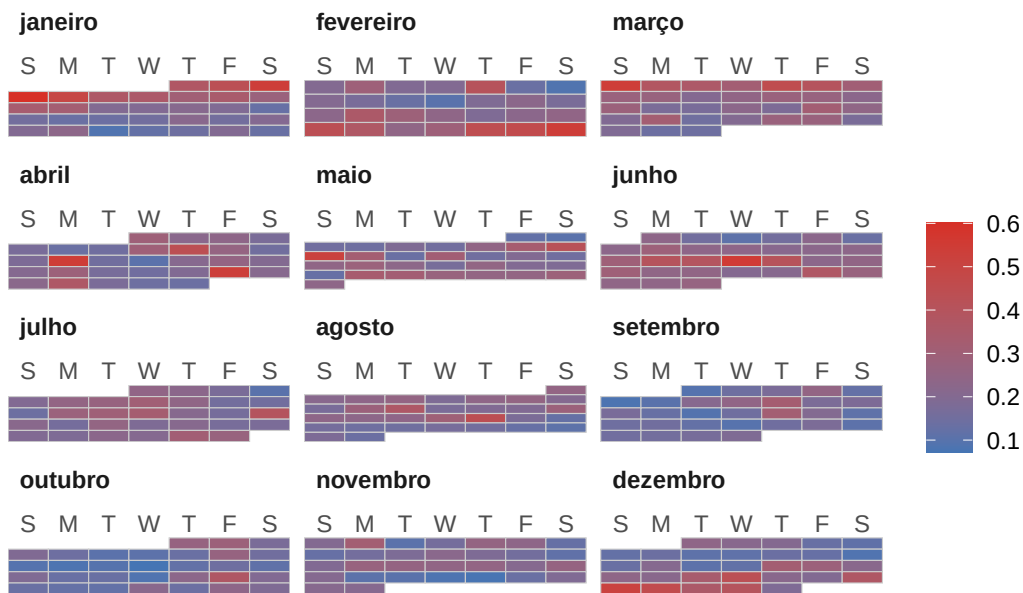
sub_stats <- stats %>% filter(Cia == cia)
ggcal(sub_stats$Data, sub_stats$Perc)
}

CAA <- baseCalendario(stats_finais, "AA")
CDL <- baseCalendario(stats_finais, "DL")
CUA <- baseCalendario(stats_finais, "UA")
CUS <- baseCalendario(stats_finais, "US")

CAA + pal + ggtitle("Atrasos na Chegada - American Airlines (AA)")

```

Atrasos na Chegada – American Airlines (AA)

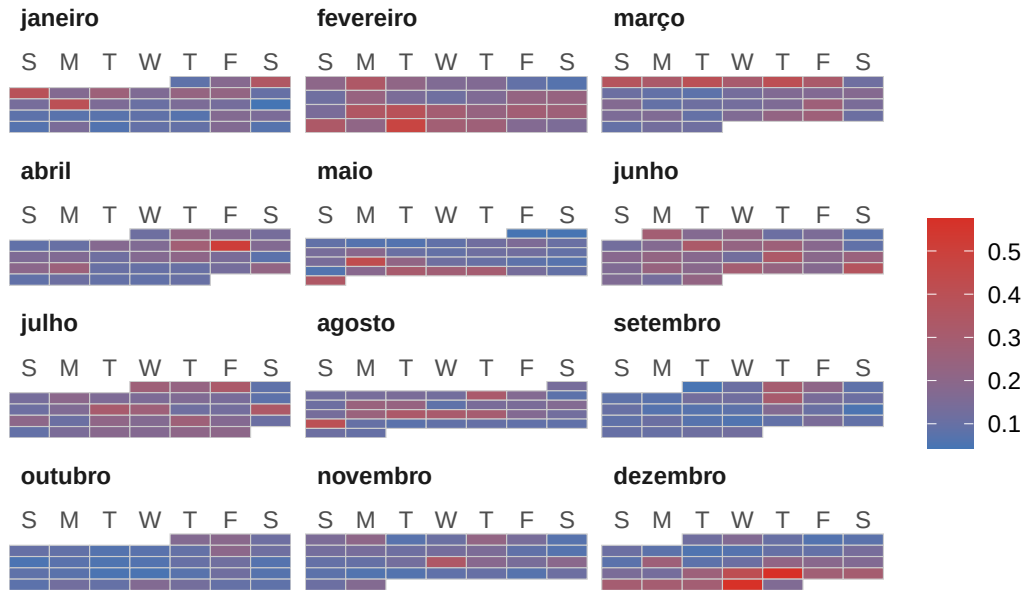


```

CDL + pal + ggtitle("Atrasos na Chegada - Delta Air Lines (DL)")

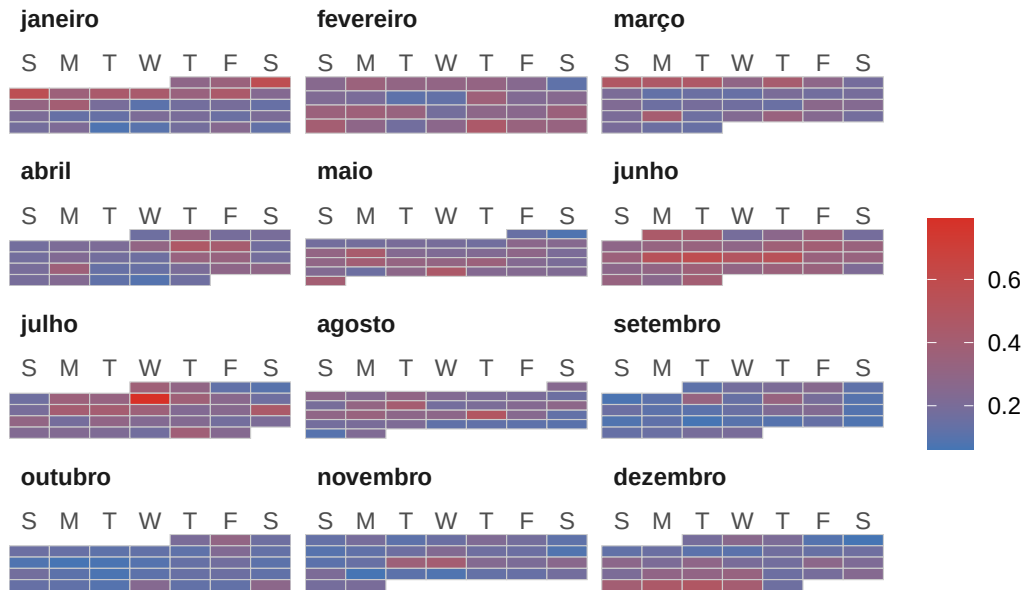
```

Atrasos na Chegada – Delta Air Lines (DL)



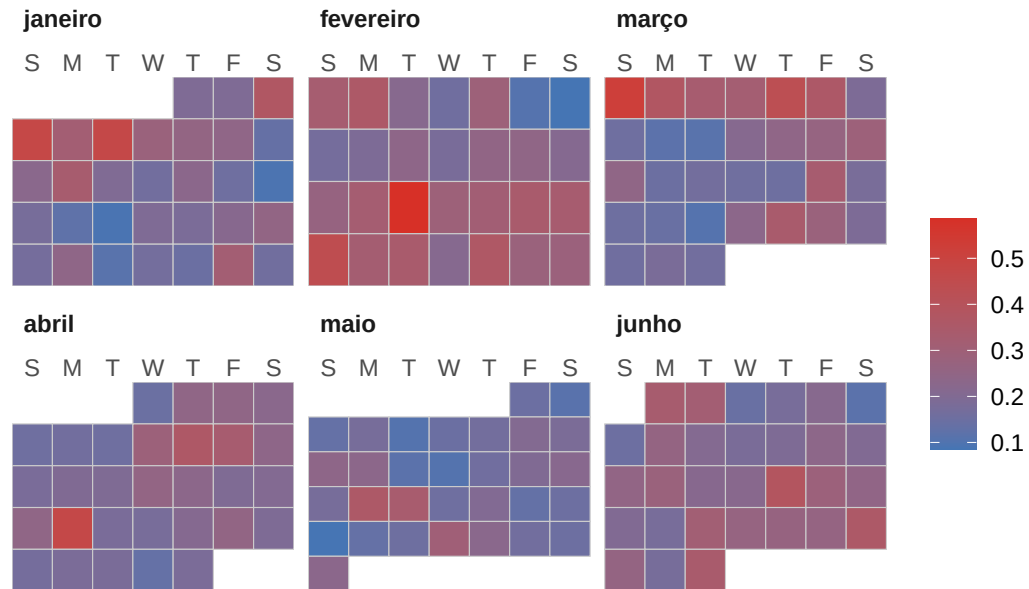
```
CUA + pal + ggtitle("Atrasos na Chegada - United Airlines (UA)")
```

Atrasos na Chegada – United Airlines (UA)



```
CUS + pal + ggtitle("Atrasos na Chegada - US Airways (US)")
```

Atrasos na Chegada - US Airways (US)



```
Sys.setenv(TZ="America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 13:16:29 -03"
```